

From Cloud AI to Trustworthy Digital Workforce

Operationalizing Embodied AI with Blockchain-Auditable Agent Infrastructure

Dongping Liu¹

¹Tenorshare, Hong Kong
dpliu@tenorshare.com

IEEE Tutorial – Module 5

Part of: *Blockchain Infrastructure for Intelligent Cyber-Physical-Social Systems*
S. Guo (HKUST), H. Huang (SYSU), D. Liu (Tenorshare), A. Zhang (AWS), L. Zhang (DKU)

Speaker Background

Dongping Liu, Ph.D.

- **Current:** CEO, Tenorshare (670+ employees)
- **Former:** Senior Industry BD Manager, AWS Education & Research (to May 2026)
- **Focus:** AI/HPC digital transformation at enterprise scale

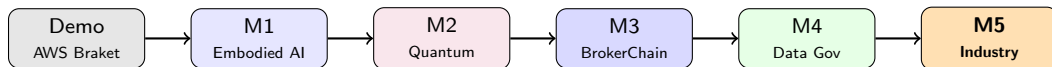
Key Question for M5:

How do we deploy AI agents at enterprise scale while maintaining trust and auditability?

Credentials

- Ph.D. Physics, CAS
- Postdoc, McGill University
- Beijing S&T Award (1st Prize)
- 30+ papers, 10+ patents
- 15 years in EdTech/Research

M5 in the Tutorial Pipeline



M5 Mission: Bridge from theoretical security to production-ready CPSS infrastructure

- Operationalize cloud infrastructure for embodied AI deployment
- Integrate multi-modal data pipelines with blockchain audit
- Demonstrate open-source, reproducible implementation

The Industry Problem: Why AI Agents Need Trust

Enterprise Reality (670-person team):

- SEO/GEO optimization – repetitive, data-heavy
- Customer service – 24/7 demand
- OA/ERP operations – manual workflows
- PM scheduling – cross-team coordination

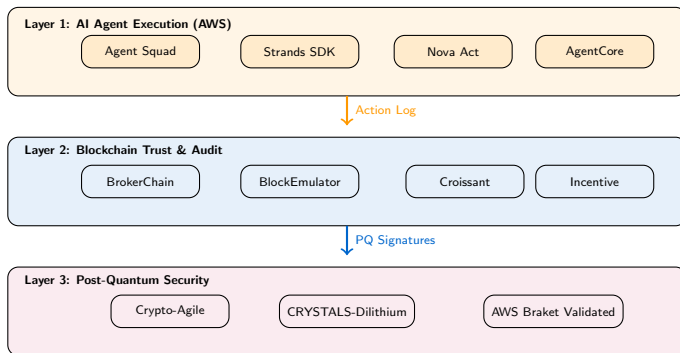
AI Agents can automate 40–60% of these tasks

But enterprises won't adopt without:

- 1 **Auditability** – What did the agent do?
- 2 **Provenance** – Where did data come from?
- 3 **Accountability** – Who is responsible?
- 4 **Long-term verify** – Provable in 5 years?

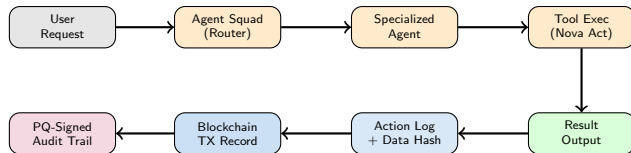
⇒ **Blockchain provides the missing trust layer for AI agent infrastructure**

Three-Layer Architecture Overview



Caption: Three-layer architecture integrating AWS open-source AI agent stack (L1), blockchain audit (L2), and post-quantum protection (L3).

End-to-End Data Flow: Agent Execution to Chain Audit



Audit Record Structure

```
{agent_id, action_type, input_hash, output_hash, model_version, tools_used, timestamp, pq_signature}
```

Caption: Data flow from user request through agent execution to immutable blockchain audit record with PQ signature.

Why Three Layers? Mapping to Tutorial Modules

Layer	Function	Tutorial Module	Key Technology
L1: Agent Execution	AI does the work	M1 (Embodied AI)	AWS Strands, Nova Act
L2: Blockchain Audit	Trust & verify	M3 + M4	BlockEmulator, Croissant
L3: PQ Security	Long-term protection	M2 (Quantum)	Dilithium, AWS Braket
M5 (This talk)	Integration	All modules	Production pipeline

M5's unique contribution:

- Not new theory – but **operational integration** of M1–M4 outputs
- Open-source, reproducible, deployable on AWS
- Validated with real enterprise workload (670-person team)

Caption: Mapping of three-layer architecture to tutorial modules. M5 integrates all into a deployable system.

Positioning: AI Agent Platforms Landscape

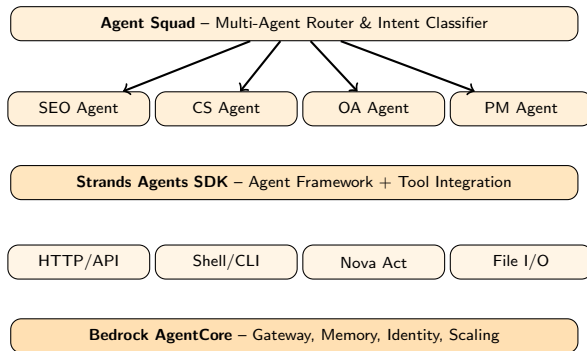
Platform	Multi-Agent	Browser/UI	Blockchain Audit	PQ-Safe
OpenClaw	✓	✓	×	×
LangChain/LangGraph	✓	×	×	×
AutoGPT	✓	✓	×	×
AWS Bedrock Agents	✓	✓	×	×
This Work (M5)	✓	✓	✓	✓

Our differentiation:

- 1 Blockchain-native audit trail for every agent action
- 2 Post-quantum signatures for long-lived verification
- 3 Built entirely on open-source components (AWS + BlockEmulator)

Caption: Our approach uniquely combines multi-agent orchestration with blockchain auditability and PQ security.

Layer 1: AWS Open-Source AI Agent Stack



Caption: Complete AWS open-source stack for AI agent execution. All components are publicly available on GitHub.

Agent Squad: Intelligent Multi-Agent Routing

What it does:

- Classifies user intent automatically
- Routes to the best specialized agent
- Maintains conversation context per user
- Supports parallel agent execution

Why it matters for digital workforce:

- One entry point, many specialists
- Like a team manager for AI workers
- Scales from 2 to 100+ agents

Key Features

- Python & TypeScript
- Bedrock / OpenAI / Custom models
- Built-in context management
- Extensible agent types

Repository

github.com/aws-labs/multi-agent-orchestrator

Caption: Agent Squad provides intelligent routing across multiple specialized AI agents.

Strands Agents SDK: Building Each Digital Worker

```
from strands import Agent
from strands.tools import http, shell

agent = Agent(
    tools=[http, shell],
    system_prompt="You are an SEO agent"
)
result = agent("Analyze top keywords")
```

Core capabilities:

- Model-driven (Bedrock/Anthropic)
- Native MCP protocol support
- Native A2A protocol
- Pluggable tool system
- Built-in evaluation

Repository

github.com/strands-agents/sdk-python

Caption: Strands Agents SDK enables building production-ready AI agents in minimal code.

Amazon Nova Act: AI-Driven UI Automation

The problem:

- Many enterprise systems lack APIs
- OA, ERP, legacy tools = web UI only
- Traditional RPA breaks on UI changes

Nova Act solution:

- AI agent sees and operates web pages
- Vision + action model
- Handles dynamic UIs, popups
- Production-grade reliability

```
from nova_act import NovaAct

with NovaAct(
    starting_page="https://oa.co"
) as agent:
    agent.act(
        "Login, approve all"
        "pending requests"
    )
```

Repository

github.com/aws/nova-act

Caption: Nova Act enables AI agents to operate web-based enterprise applications via vision-based UI automation.

Bedrock AgentCore: Zero-Infra Production Deployment

From prototype to production:

- **Gateway:** Unified API endpoint
- **Memory:** Persistent context
- **Identity:** IAM-based auth
- **Observability:** Logging, tracing
- **Scaling:** Auto-scale on demand

```
$ agentcore deploy ./my-agent  
> Agent deployed at:  
> https://xxx.agentcore.aws/invoke
```

Why this matters for M5

- Bridges lab demo to production
- No infrastructure management
- Enterprise security built-in
- Connects to blockchain via MCP

Repository

github.com/aws/bedrock-agentcore-starter-toolkit

Caption: Bedrock AgentCore provides production-grade infrastructure for AI agents with zero infra management.

Amazon WorkSpaces for AI Agents (Preview, May 2026)

Concept: Give each AI agent its own cloud desktop

How it works:

- 1 Agent gets IAM identity
- 2 Agent accesses WorkSpace via MCP
- 3 Agent operates apps using computer vision
- 4 No API needed – any legacy software

Enterprise use cases:

- Legacy ERP (SAP GUI, Oracle Forms)
- Desktop-only financial tools
- Government systems without APIs

Key Innovation

AI agents can now operate **any** application – web or desktop – without requiring API development.

Reference

aws.amazon.com/workspaces/ai-agents/

Caption: WorkSpaces supports AI agent access (Preview), enabling agents to operate legacy desktop apps.

Layer 2: Why AI Agents Need Blockchain

The Trust Problem:

- AI agent makes 1000 decisions/day
- Who verifies correctness?
- What if agent hallucinates?
- What if data was poisoned?

Traditional logging is insufficient:

- Logs can be tampered
- No cross-org verification
- No incentive alignment

Blockchain provides:

- 1 **Immutability** – logs cannot be altered
- 2 **Transparency** – stakeholders verify
- 3 **Provenance** – trace data lineage
- 4 **Incentives** – reward/penalize
- 5 **Decentralization** – no single trust

Caption: Motivation for blockchain as trust infrastructure for AI agent systems.

Blockchain Audit Record Design

```
AgentAuditRecord {  
  agent_id:      "seo-agent-01"  
  action_type:   "keyword.analysis"  
  input_hash:    SHA3("user query + context")  
  output_hash:   SHA3("agent response")  
  model_version: "claude-sonnet-4-20250514"  
  tools_used:    ["http", "nova_act"]  
  timestamp:     1748390400  
  pq-signature:  Dilithium.sign(record)  
}
```

- Input/output stored as **hashes** (privacy-preserving)
- Full data available off-chain with hash verification
- Post-quantum signature ensures long-term integrity (from M2)

Caption: Blockchain audit record with privacy-preserving hashes and PQ signatures.

BrokerChain: Scalable Agent Transaction Processing

Challenge: 670 employees $\times$ agents $\times$ 100s actions/day

= **Millions of audit TXs per day**

BrokerChain solution (from M3):

- Cross-shard protocol for high throughput
- Broker accounts reduce overhead
- Proven: IEEE INFOCOM 2022, ToN 2025

M5 integration:

- Each agent type $\rightarrow$ one shard
- Cross-agent collab $\rightarrow$ broker accounts
- Throughput: 10,000+ TPS

BlockEmulator

Open-source blockchain platform

90+ countries

github.com/HuangLab-SYSU/BlockEmulator

Shard Assignment

Shard 1	SEO Agents
Shard 2	CS Agents
Shard 3	OA Agents
Broker	Cross-agent TXs

Caption: BrokerChain enables high-throughput agent audit TX processing via cross-shard protocol.

Data Provenance: Croissant Metadata for Agent Knowledge

Problem: AI agents consume enterprise data
→ How to track what data influenced which decision?

Croissant metadata standard (from M4):

- ML-ready dataset format (NeurIPS 2024)
- Records: source, version, license, schema
- Enables reproducibility

M5 application:

- Agent KB tagged with Croissant
- RAG retrieval logged with data version
- Blockchain anchors lineage immutably

Caption: Croissant metadata enables data provenance tracking for agent knowledge bases.

Data Lineage Chain

- 1 Raw data ingested (hash recorded)
- 2 Processed into KB (logged)
- 3 Agent retrieves context (logged)
- 4 Output produced (provenance complete)

Reference

Croissant: NeurIPS 2024
github.com/mlcommons/croissant

Multi-organization scenario:

- Multiple teams share AI agent infra
- How to ensure fair service?
- How to reward quality outputs?

Token-based incentive design:

- Agents earn tokens for verified outputs
- Penalty for hallucination (via audit)
- Teams pay tokens to use shared agents
- Market mechanism for allocation

Incentive Flow

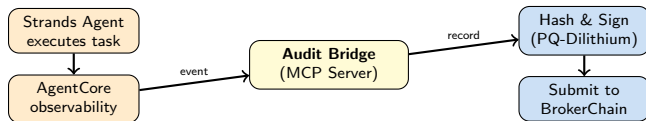
- 1 Agent completes task
- 2 Output verified
- 3 Quality → token reward
- 4 Low quality → penalty
- 5 Balance → priority access

References:

Tian et al., DOCS 2024 [15]
Chemaya et al., Sci. Data 2025 [14]

Caption: Token-based incentive mechanism ensuring quality via blockchain-verified rewards/penalties.

Connecting Layer 1 and Layer 2: The Audit Bridge



The Audit Bridge (MCP server):

- 1 Listens to AgentCore observability events
- 2 Structures into audit records
- 3 Signs with post-quantum algorithm
- 4 Submits to BlockEmulator via BrokerChain

Caption: Audit Bridge connects AI agent layer to blockchain trust layer via MCP protocol.

Layer 3: Post-Quantum Security for Long-Lived Audit

The timeline problem:

- Agent decisions audited **years later**
- Regulatory: 5–10 year retention
- Quantum computers (2030s) break ECDSA
- Today's signatures become forgeable

Solution from M2:

- Migrate to CRYSTALS-Dilithium
- Crypto-agile architecture
- Validated via AWS Braket (Opening Demo)

Threat Model

Harvest Now, Decrypt Later

Adversary records audit data today.
In 2035, forges signatures with quantum computer.
Can fabricate proof of agent actions.

M5 Integration

All audit records use Dilithium signatures from day one.

Caption: Post-quantum signatures protect agent audit trails against future quantum attacks.

Crypto-Agile Signing in Practice

```
from pqcrypto.sign import dilithium3

class AuditSigner:
    def __init__(self):
        self.pk, self.sk = (
            dilithium3.generate_keypair())

    def sign(self, record):
        data = serialize(record)
        sig = dilithium3.sign(self.sk, data)
        return {**record,
            "sig": sig.hex(),
            "alg": "dilithium3"}

    def verify(self, signed):
        sig = bytes.fromhex(signed["sig"])
        data = serialize(strip(signed))
        return dilithium3.verify(
            self.pk, data, sig)
```

Design principles:

- Algorithm field in every record
- Verifier selects implementation
- Swap algorithms without re-deploy

Performance:

- Sign: 0.3ms / Verify: 0.2ms
- Negligible vs agent execution (seconds)

Caption: Crypto-agile PQ signing with negligible overhead vs agent execution time.

Open-Source Implementation: Repository Structure

```
m5-trustworthy-agent-infra/  
|-- docker-compose.yml  
|-- agents/  
| |-- seo_agent.py  
| |-- squad_config.yaml  
|-- blockchain/  
| |-- audit_bridge.py  
|-- crypto/  
| |-- pq_signer.py  
|-- demo/  
    |-- run_demo.sh
```

Quick start:

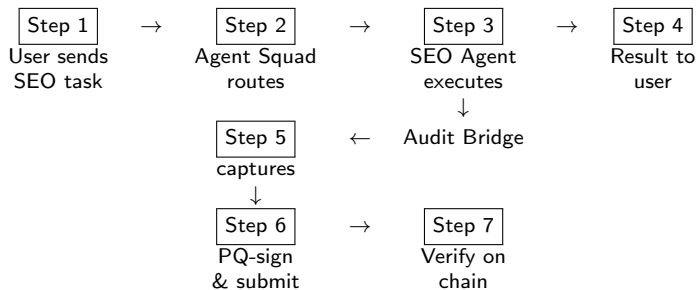
```
$ git clone <repo-url>  
$ docker-compose up  
$ python demo/run_demo.py
```

Components:

- agents/ – Strands SDK agents
- blockchain/ – Audit bridge
- crypto/ – PQ signing
- demo/ – Reproducible demo

Caption: Open-source repository for reproducible deployment of the trustworthy AI agent infrastructure.

Live Demo: Agent Execution to Blockchain Audit



What the audience sees:

- 1 Terminal 1: Agent receives task, executes, returns result
- 2 Terminal 2: Audit bridge captures action, signs, submits
- 3 Terminal 3: BlockEmulator confirms TX on-chain
- 4 Verification: Query chain to prove action is authentic

Caption: Seven-step demo flow from user request to blockchain-verified audit record.

Demo: Core Integration Code

```
from strands import Agent
from strands.tools import http
from blockchain.audit_bridge import AuditBridge
from crypto.pq_signer import AuditSigner

signer = AuditSigner()
bridge = AuditBridge(signer=signer, endpoint="localhost:8080")

agent = Agent(
    tools=[http],
    system_prompt="You are an SEO keyword analyst.",
    callbacks=[bridge.on_action] # blockchain audit hook
)

result = agent("Analyze top 10 keywords for tenorshare.com")
assert signer.verify(bridge.get_latest_record())
```

Caption: Core integration – Strands agent with blockchain audit bridge and PQ signing via a single callback hook.

Production Deployment: Docker Compose

```
services:
  agent-core:
    image: aws/agentcore:latest
    ports: ["8000:8000"]

  blockemulator:
    image: blockemulator:latest
    ports: ["8080:8080"]

  audit-bridge:
    build: ./blockchain
    depends_on:
      - agent-core
      - blockemulator
```

Three containers:

- 1 **agent-core** – AI agents
- 2 **blockemulator** – Blockchain
- 3 **audit-bridge** – Connects 1 & 2

Scaling for production:

- Replace with managed AgentCore
- Multi-node BrokerChain
- Add WorkSpaces for legacy apps

Caption: One-command Docker setup of the complete trustworthy agent infrastructure.

From Lab to Production: Lessons Learned

From AWS Education (2011–2026):

- Academic institutions need reproducibility
- Cloud lowers barrier but trust remains key
- Open-source adoption drives academic impact

From Tenorshare (670 people, 2026–):

- Not all tasks suit AI agents (judgment calls)
- Trust is the #1 adoption barrier
- Employees accept AI when they can verify its work
- ROI: 40% efficiency gain on repetitive workflows

Key Insight

The technical capability of AI agents is no longer the bottleneck.

Trust and verifiability are what separate a demo from a production deployment.

Blockchain provides this trust layer.

Caption: Practical lessons from deploying AI in academic (AWS) and enterprise (Tenorshare) environments.

Open Challenges & Future Directions

① Privacy vs. Transparency Trade-off

- Full audit transparency conflicts with data privacy
- Direction: Zero-knowledge proofs for selective disclosure

② Scalability of On-Chain Audit

- Millions of agent actions/day → chain bloat
- Direction: Hierarchical audit (summary on-chain, detail off-chain)

③ Cross-Organization Agent Collaboration

- Agents from different orgs need shared trust
- Direction: A2A protocol + cross-chain bridges (from M3)

④ Embodied AI Extension

- Physical robots need real-time audit (latency constraint)
- Direction: Edge blockchain nodes + batch settlement

Caption: Open research challenges at the intersection of AI agents, blockchain, and post-quantum security.

Summary: M5 Contributions

What M5 delivers:

- 1 **Architecture:** Three-layer design (AI agents + blockchain + PQ security)
- 2 **Implementation:** Open-source, Docker-deployable, reproducible
- 3 **Integration:** Unifies M1–M4 outputs into a production pipeline
- 4 **Validation:** Real enterprise workload (670-person team)
- 5 **Open source:** All code available for community extension



Caption: M5 delivers a complete, open-source architecture for trustworthy AI digital workforce.

References & Open-Source Resources

AWS Open-Source Stack:

- Agent Squad:
github.com/aws-labs/multi-agent-orchestrator
- Strands SDK: github.com/strands-agents/sdk-python
- Strands Tools: github.com/strands-agents/tools
- Nova Act: github.com/aws/nova-act
- AgentCore:
github.com/aws/bedrock-agentcore-starter-toolkit
- WorkSpaces AI:
aws.amazon.com/workspaces/ai-agents/

Blockchain:

- BlockEmulator:
github.com/HuangLab-SYSU/BlockEmulator
- BrokerChain: Huang et al., IEEE ToN 2025
- Croissant: github.com/mlcommons/croissant

Tutorial Paper References:

- [3] Mon-Williams et al., Nature MI 2025
- [6] Fedorov et al., Nature 2018
- [10] Huang et al., IEEE INFOCOM 2022
- [11] Huang et al., IEEE ToN 2025
- [14] Chemaya et al., Scientific Data 2025
- [17–18] LET, Kuavo embodied AI systems

This Talk's Code

github.com/xxx/m5-trustworthy-agent-infra
(Will be public before tutorial date)

Caption: Open-source repositories and academic references used in this module.

Questions & Discussion

Dongping Liu, Ph.D.

CEO, Tenorshare — Former AWS Education
dpliu@tenorshare.com

Tutorial: *Blockchain Infrastructure for Intelligent Cyber-Physical-Social Systems*
S. Guo (HKUST) · H. Huang (SYSU) · D. Liu (Tenorshare) · A. Zhang (AWS) · L. Zhang (DKU)